

E-Commerce Hybrid Data Pipeline

Event-Driven Streaming & Batch Data Platform



1. 프로젝트 개요

01

Olist 데이터셋 기반

실제 이커머스 데이터를 활용한 엔지니어링 프로젝트

02

이벤트 기반 재설계

정적 CSV → Event-Driven Streaming 구조로 전환

03

Hybrid Pipeline

Kafka · Spark · Airflow · MinIO 기반 플랫폼 구축

04

Data Lake & Data Mart 아키텍처 구축

Bronze -> Silver -> Gold 계층 설계

2. 사용 데이터셋 & 이벤트 모델링

사용 데이터

Olist Dataset

- Orders
- Customers
- Order Items
- Products
- Reviews

Additional

Nemotron Persona
Korea Data

이벤트 모델링

Order Events

ORDER_CREATED
ORDER_APPROVED
ORDER_CANCELED

Delivery Events

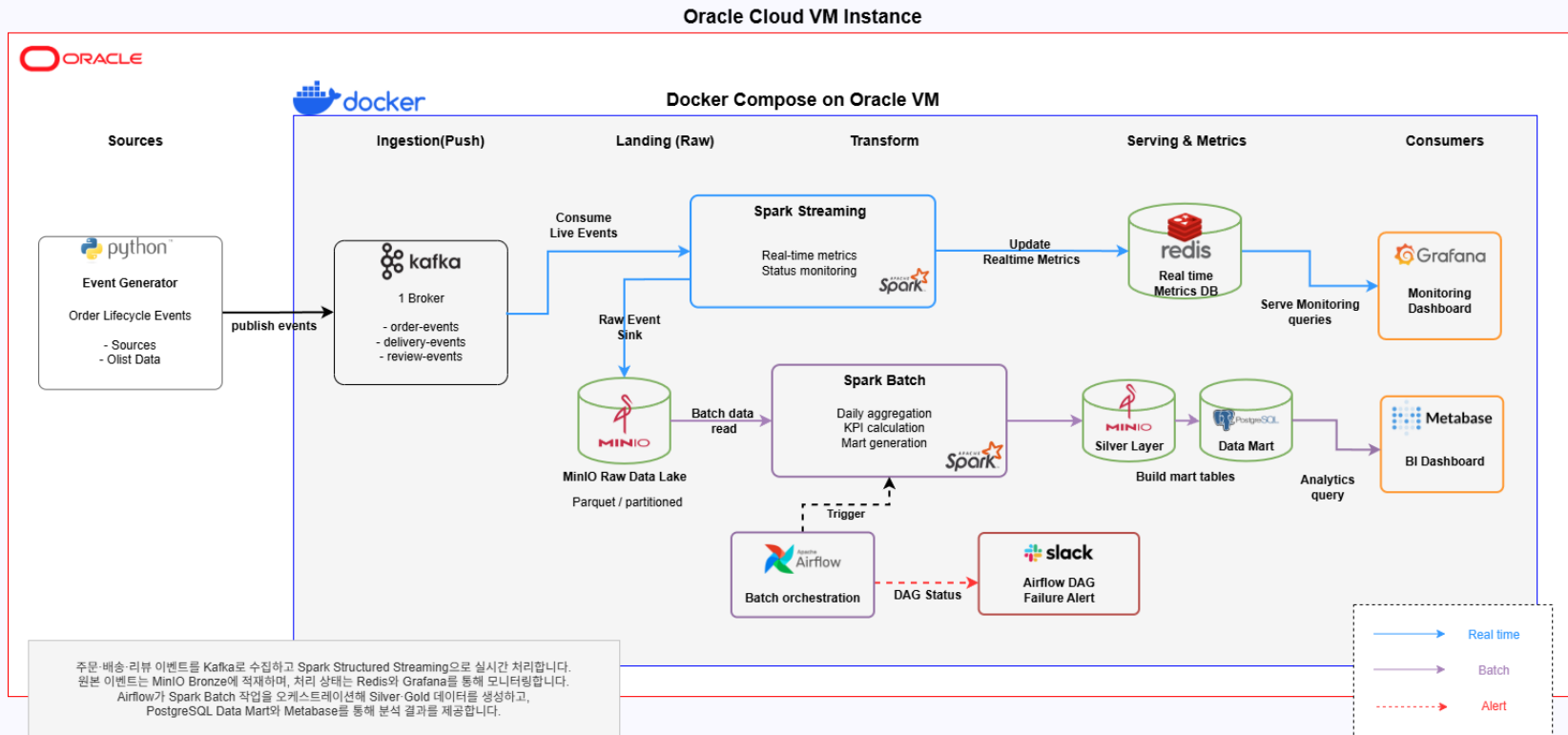
DELIVERY_STARTED ·
DELIVERY_COMPLETED

Review Events

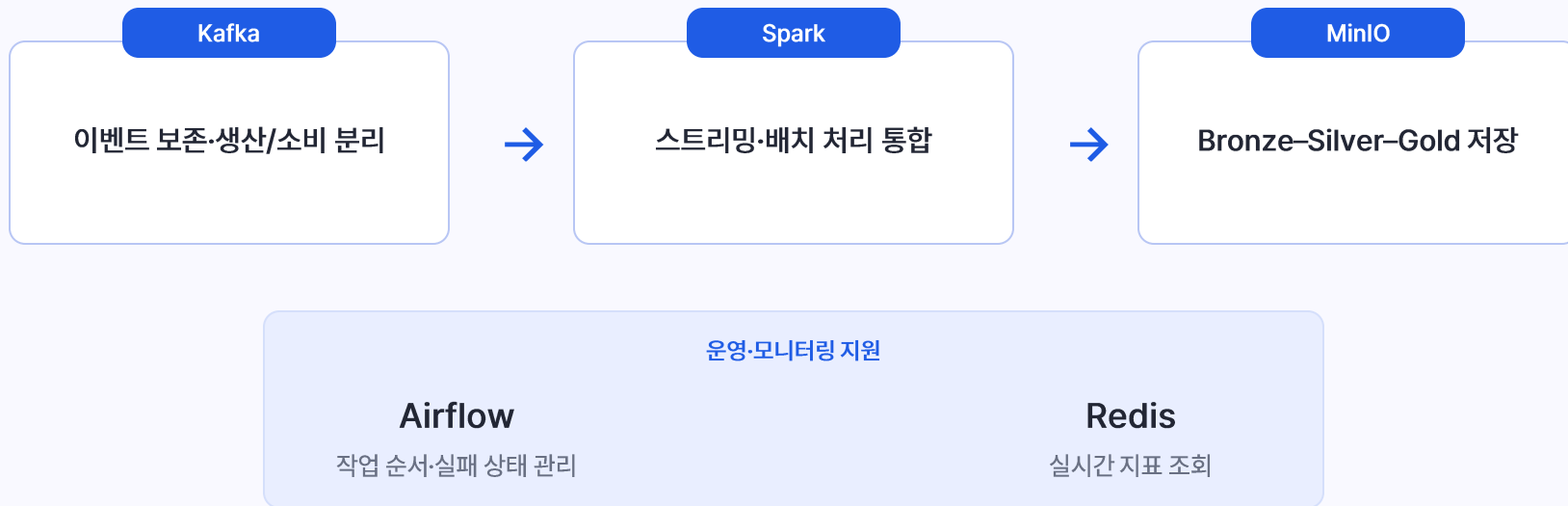
REVIEW_CREATED

① 핵심 설계 원칙: 정적 Row 데이터를 시간 흐름 기반 Event Stream 구조로 변환하여 실시간 처리 가능하도록 재모델링

3. 전체 아키텍처

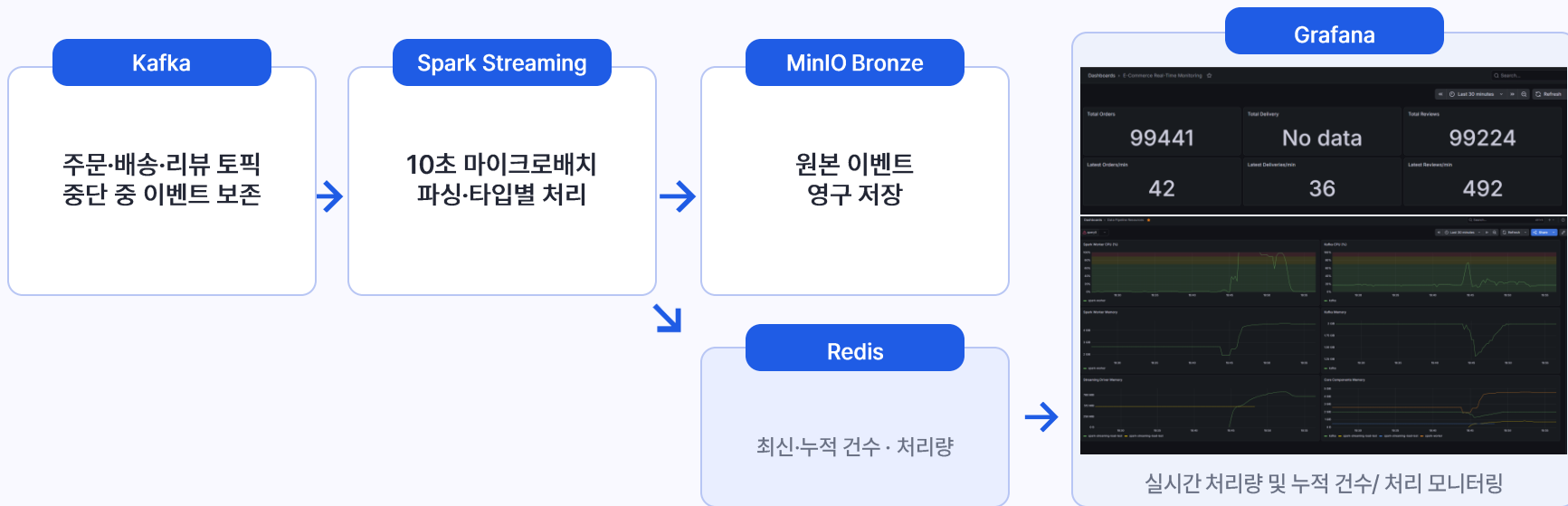


4. 아키텍처 세부사항



데이터 규모보다 스트리밍·배치를 하나의 처리 모델로 구성하고, 복구 가능한 구조를 검증하기 위해 Spark를 선택

5. 스트리밍 처리



컨테이너 재시작 시 체크포인트와 Kafka Offset을 기준으로 미처리 이벤트부터 이어서 처리

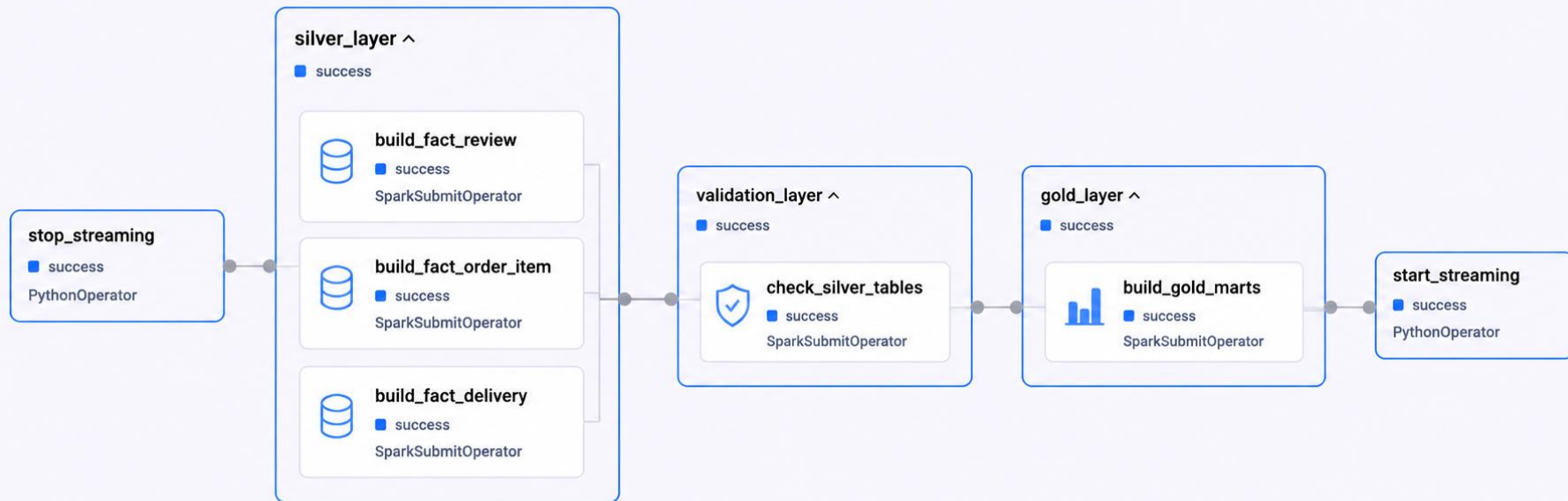
6. 배치 처리 – Bronze → Silver → Gold 레이어



스토리지 전략

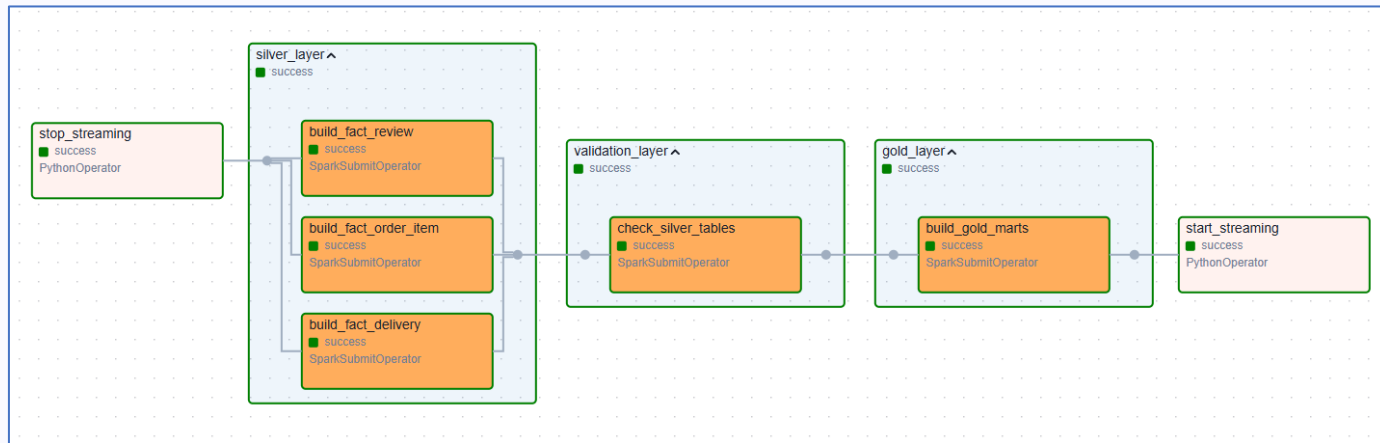
저장소	용도	
 MinIO	Raw / Parquet / Data Lake	✓ Parquet 포맷 + 파티셔닝으로 쿼리 성능 향상 MinIO를 통해 비용 효율적인 Data Lake를 구현
 PostgreSQL	Fast SQL Query / BI	

6. 배치 처리 – Airflow 워크플로우



리소스 부족 -> kafka 이벤트 보존 및 spark 체크포인트 활용하여 배치시간 동안 스트리밍 중단

6. 배치 처리 – Airflow 워크플로우



data-pipeline-alert
1명의 멤버 • 2개의 탭

Airflow Task Failed
DAG: ecommerce_batch_pipeline
Task: silver_layer.build_fact_review
Execution Date: 2026-05-18 11:18:21.101562+00:00
Log: http://localhost:8080/dags/ecommerce_batch_pipeline/grid?dag_run_id=manual_2026-05-18T11%3A18%3A21.101562%2B00%3A00&task_id=silver_layer.build_fact_review&base_date=2026-05-18T11%3A18%3A21%2B0000&tab=logs

Airflow Alert Bot 11:38 오후

Airflow Task Success
DAG: ecommerce_batch_pipeline
Task: gold_layer.build_gold_marts
Execution Date: 2026-05-18 13:59:46.196667+00:00
Log: http://localhost:8080/dags/ecommerce_batch_pipeline/grid?dag_run_id=manual_2026-05-18T13%3A59%3A46.196667%2B00%3A00&task_id=gold_layer.build_gold_marts&base_date=2026-05-18T13%3A59%3A46%2B0000&tab=logs

7. 데이터 모델링



분석 기준 Fact·Dimension → 목적별 Mart → PostgreSQL → Metabase

8. 트러블 슈팅

	문제	핵심 해결	결과
01	실시간 지표 누락	processing_time 기준 변경	처리량 정상 집계
02	체크포인트 저장 실패	MinIO 경로·권한 정리	상태 복구 정상화
03	배치·스트리밍 자원 경쟁	중지 → 배치 → 재시작	배치 후 스트리밍 재개
04	스트리밍 컨테이너 재시작	Raw 단계의 dropDuplicates 제거 및 체크포인트 분리	재시작 안정화 →

8. 트러블 슈팅 상세

BEFORE



여러 Driver 지표 중첩 · 메모리 사용량 상승

문제

Query 종료가 완료되기 전에
컨테이너가 반복 재시작

원인

dropDuplicates State Store와
체크포인트 상태 불일치

AFTER



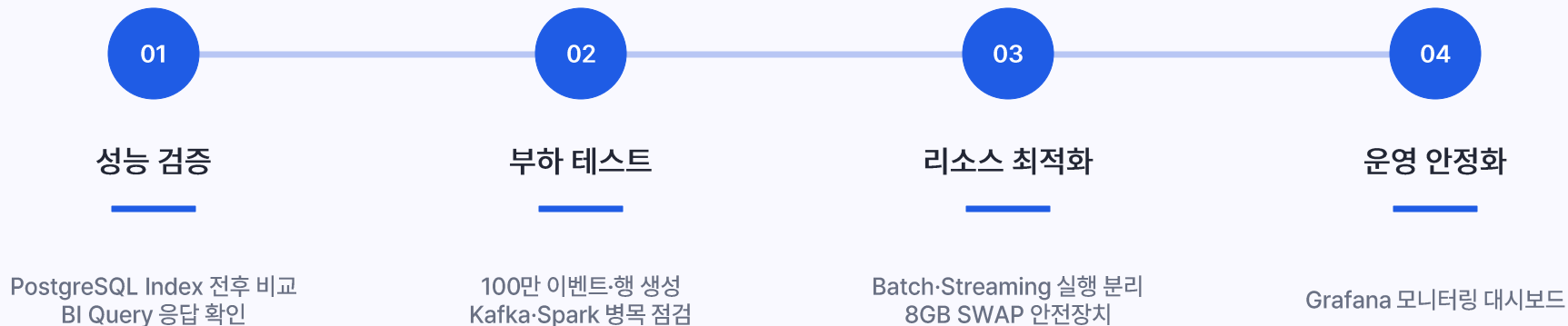
실행 상태 정리 · 재시작 후 지표 안정화

해결

Raw 단계의 dropDuplicates 제거
및 체크포인트 분리

9. 프로젝트 개선

프로젝트 진행 중 확인한 보완 과제



성능 측정 → 병목 확인 → 자원 안정화 → 운영 가시성 확보

10. 개선 결과 및 검증

1,000,000

EVENTS

Kafka → Spark → MinIO 정상 적재

10,000

EPS

최대 입력 설정에서 흐름 검증

144 x

MAX

조회 유형별 3.5~144배 개선

부하 테스트 리소스 모니터링



PostgreSQL 인덱스 적용 효과

적용 전

```
parallel (parallel 100, 100, 100) 100 rows (100) width=100 (actual time=1.342..17.211 msec)
Worker Launched: 2
Buffer shared hit=7120
=> Parallel Seq Scan on parking_ticket_items (cost=0.00..8756.58 rows=481 width=218)
Filter (product_id = 'a3a3b7d00e1a7b0d0e0014001a7a0e01')
Rows Removed by Filter: 16884
Buffer shared hit=7120
Planning Time: 0.000 ms
Execution Time: 22.321 ms
```

Parallel Seq Scan · 22.321 ms

적용 후

```
parallel (parallel 100, 100, 100) 100 rows (100) width=100 (actual time=1.342..17.211 msec)
Worker Launched: 2
Buffer shared hit=7120
=> Parallel Seq Scan on parking_ticket_items (cost=0.00..8756.58 rows=481 width=218)
Filter (product_id = 'a3a3b7d00e1a7b0d0e0014001a7a0e01')
Rows Removed by Filter: 16884
Buffer shared hit=7120
Planning Time: 0.000 ms
Execution Time: 22.321 ms
```

Bitmap Index Scan · 1.200 ms

안정성 보완 및 검증, 조회 성능 개선

체크포인트 재개 · 8GB SWAP · Load Test 자동화 · PostgreSQL Index 적용

측정 한계

Producer가 11,000 EPS 부근에서 선행 병목발생 -> Spark 최대 처리량이 아닌 입력 값으로 10,000 EPS 설정

11. 결론

구축 성과

- 스트리밍·배치 통합
- 계층형 저장과 BI Serving
- 체크포인트 기반 재개
- 단일 VM 자원 제어

확인된 한계

- Producer 병목으로 Spark 한계 미 측정
- 분산 확장 효과 검증 제한
- 생성 데이터와 실제 트래픽 차이

다음 개선

- Producer 병렬화 후 재측정
- 데이터 품질·실패 데이터 격리
- 스키마 변경 감지·알림 확대

수집·저장·변환·복구·모니터링이 연결된 운영 구조를 직접 구현하고 검증

Thank You

Q & A

